



Systems Engineering

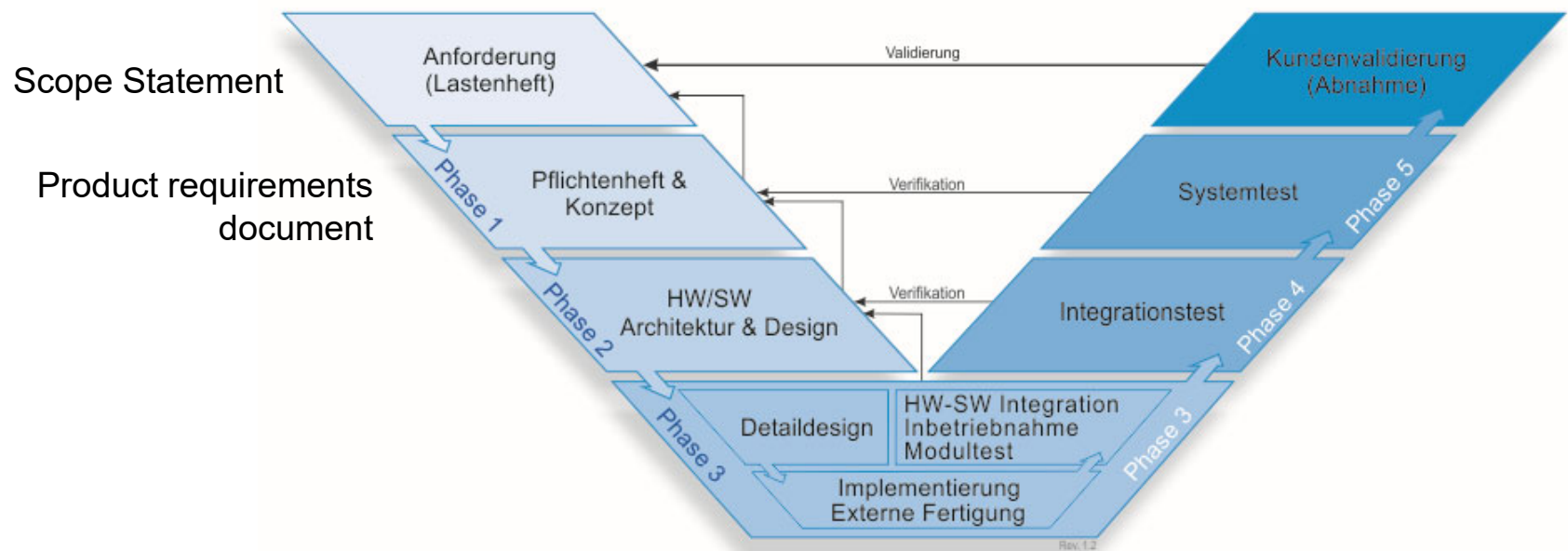
Chapter 8 – Formal Verification

Prof. Dr. Kai Höfig



Verification and Validation

- ♦ **Verification** is a proof, that a given program or system is behaving correctly according to its *specification*.
- ♦ **Validation** is an assessment of qualification of a software or a system according to its intended use (customer expectations).



<https://www.mesco-engineering.com/de-de/leistungen/embedded-softwareentwicklung/methodik>



Requirements for sound verification techniques

Good verification technologies are

- ◆ Fast .. Do not slow down significantly the development process
- ◆ Cheap .. Do not increase significantly the costs of the process
- ◆ Useable .. Can be executed by systems or software engineers
- ◆ Integrated .. Can be applied during the development, not only at the end
- ◆ Safe .. Do not insert new errors to the development
- ◆ Automated .. Do not require a lot of manual work
- ◆ Exhaustive .. After applying, system behaves according to specification
- ◆ Uncovering .. Do not only show conformity, but also shows where an error is located by giving a counterexample
- ◆ Subtle .. Finds also errors for rare conditions
- ◆ Measurable .. Allows to quantify the quality of a program



Different Verification Techniques

- ◆ **Reviews** (Peer Review, Walkthrough, Inspection) – Manual task with low completeness, but easy to apply and effective during early phases of the development.
- ◆ **Emulation** – Hardware is configured in a way that it behaves like the system under test. Applied e.g. during chip development using reprogrammable chips followed by *testing*.
- ◆ **Simulation** – Model of the system is executed in software, followed by *testing*.
- ◆ **Test** – Test cases are compared to its expected behavior. Typically does not proof the correctness of a program. Can be automated. Is a creative process.
- ◆ **Formal Verification** – is at least complete according to a certain aspect, is automated, is hard to apply and costs a lot of effort.



Testing

- ◆ Goal of a test is to uncover faults, *not correctness of a system or a program*.
- ◆ Correctness can typically not be proven, since the number of inputs and internal states are either infinite or very large. Example:
 - For a simple operation that adds two integer values, this would be $65.535^2 = 4.294.836.255$ different test cases to really *proof*, that the operation is correctly implemented.
 - If every test case lasts a single second, the entire test would last 138 years.
 - Additionally, we need to know the result of all 4.294.836.255 operations to find an error (test oracle problem)





System Test

- ◆ A system test is a so-called *black-box test* against a given specification and are also called functional tests.
- ◆ Typically, the specified behavior is tested, unspecified behavior is not detected systematically (e.g. viruses in software)
- ◆ Minimal test: there is at least one test case per requirement.
- ◆ Test cases are reduced by intelligently covering more than one requirement.

white box: all lines of codes are known and executed in the test

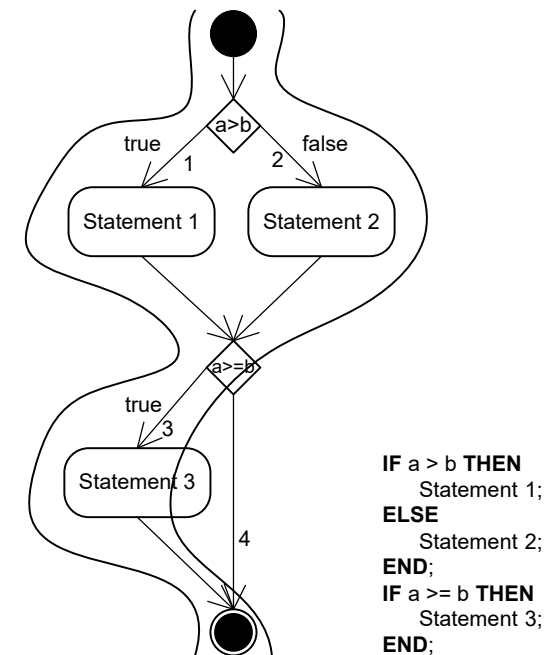
black box: output is evaluated based on requirements





White Box Testing (Software only)

- ◆ As a counterpart to black box testing, there is also white-box testing, which is applied during software development only.
- ◆ White-Box-Tests use the control or data flow of a program to measure the coverage of a number of test cases to measure the quality of tests.
- ◆ Since finding appropriate test cases is a creative process, coverage criteria support the test engineer and tell him when test cases fulfill a certain quality.
- ◆ Examples of such coverage criteria are
 - Statement Coverage C_0
 - Branch Coverage C_1
 - Path Coverage C_2
 - MCC, MC/DC ...





Formal Verification

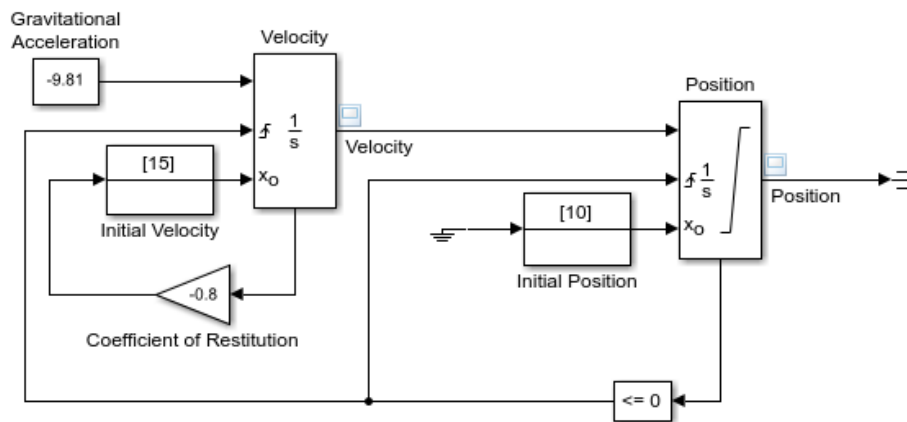
- ◆ Verification that is based on
 - Mathematical precise models
 - Mathematical precise process and rules

- ◆ Techniques
 - Model-based Simulation
 - Model-based Testing
 - Model Checking
 - Computer-assisted Proof

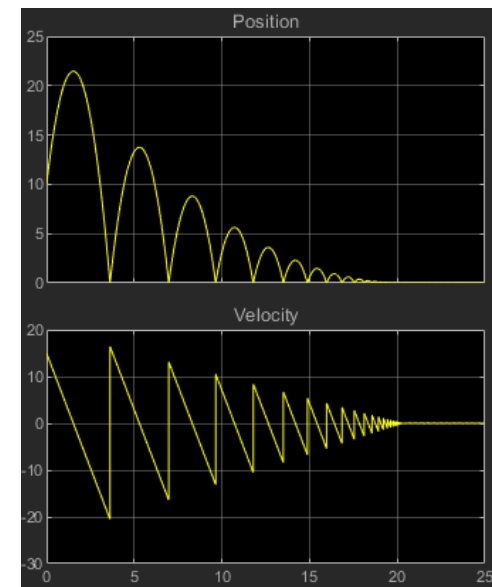


Model-based Simulation

- ◆ In model-based simulation, a software tool is used to simulate system behavior for automated or user defined scenarios. In contrast to testing, the system behavior is analyzed for an entire scenario rather than a single test case. Example: A bouncing ball in Simulink®



Formal Model



Simulation

Easy to execute
Fast
Shows counterexamples

Not a measure for quality
Only analyzes user-defined scenarios
Not exhaustive

<https://de.mathworks.com/help/simulink/slref/simulation-of-a-bouncing-ball.html>



Model-based Testing

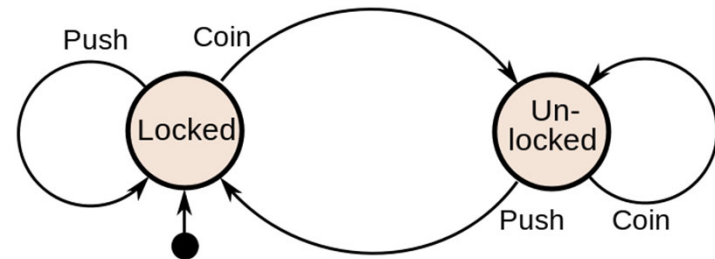
- ◆ Test-cases are automatically derived from the system model and then executed. Results are being analyzed by the correct state.

```
# Comment
@tag
Feature: Eating too many cucumbers may not be good for you

    Eating too much of anything may not be good for you.

Scenario: Eating a few is no problem
    Given Alice is hungry
    When she eats 3 cucumbers
    Then she will be full
```

*Design specification in Gherkin that
Cucumber can use to derive test cases*



Finite state machine for a turnstile

*Easy to execute
Fast
Shows counterexamples
Uses coverage criteria*

*Only executes a representative
Not exhaustive*



Computer Assisted Proof

- ◆ System behavior is modeled using logic formulas. The correct behavior is a property and the system is correct, if the correct property follows from the system formalization.
- ◆ Is often used for **software** or software-intense systems
- ◆ Example implementations
 - Coq <https://coq.inria.fr/>
Mathematical definitions, executable algorithms and theorems together with an environment for semi-interactive development of machine-checked proofs.
 - Isabelle <https://isabelle.in.tum.de/>
It allows mathematical formulas to be expressed in a formal language and provides tools for proving those formulas in a logical calculus for functional programming.
 - PVS <http://pvs.csl.sri.com/>
Environment for formal specification and verification using a specification language and an interactive theorem prover.

*Proof, freedom of faults
Can be used for systems with infinite states*

*Slow
Often manual task, since problem is undecidable
Often no solution*



Example of a Mathematical Proof in Isabelle

```
section <The Drinker's Principle>
```

```
theory Drinker
  imports Main
begin
```

```
text <Here is another example of classical reasoning: the Drinker's Principle says
  that for some person, if he is drunk, everybody else is drunk!
  We first prove a classical part of de-Morgan's law.>
```

```
lemma de_Morgan:
  assumes " $\neg (\forall x. P\ x)$ "
  shows " $\exists x. \neg P\ x$ "
proof (rule classical)
  assume " $\neg \forall x. P\ x$ "
  have " $\forall x. P\ x$ "
  proof
    fix x show "P x"
    proof (rule classical)
      assume " $\neg P\ x$ "
      then have " $\exists x. \neg P\ x$ " ..
      with < $\neg \forall x. P\ x$ > show ?thesis
        by contradiction
    qed
  qed
  with < $\neg (\forall x. P\ x)$ > show ?thesis
    by contradiction
qed
```

```
theorem Drinker's_Principle: " $\exists x. \text{drunk } x \rightarrow (\forall x. \text{drunk } x)$ "
proof cases
  assume " $\forall x. \text{drunk } x$ "
  then have " $\text{drunk } a \rightarrow (\forall x. \text{drunk } x)$ " for a ..
  then show ?thesis ..
next
  assume " $\neg (\forall x. \text{drunk } x)$ "
  then have " $\exists x. \neg \text{drunk } x$ " by (rule de_Morgan)
  then obtain a where " $\neg \text{drunk } a$ " ..
  have " $\text{drunk } a \rightarrow (\forall x. \text{drunk } x)$ "
  proof
    assume "drunk a"
    with < $\neg \text{drunk } a$ > show " $\forall x. \text{drunk } x$ " by contradiction
  qed
  then show ?thesis ..
qed
end
```

Inconvenient for complex systems...



Model Checking

- ◆ In Model checking, specified properties of a system are automatically proved to hold using a model of a system. The state space of a system has to be finite, since it is analyzed exhaustively. Because of this property, model checking can be used better for control-intensive applications rather than data-intensive applications.
- ◆ **Pro**
 - All properties can be proven individually
 - “Push Button” Technology
 - No creativity for creating test cases required
 - Covers entire logical areas instead of single test cases
- ◆ **Con**
 - All model-based methods are only as good as their model
 - State space explosion leads to non-calculability quickly



An Informal Example

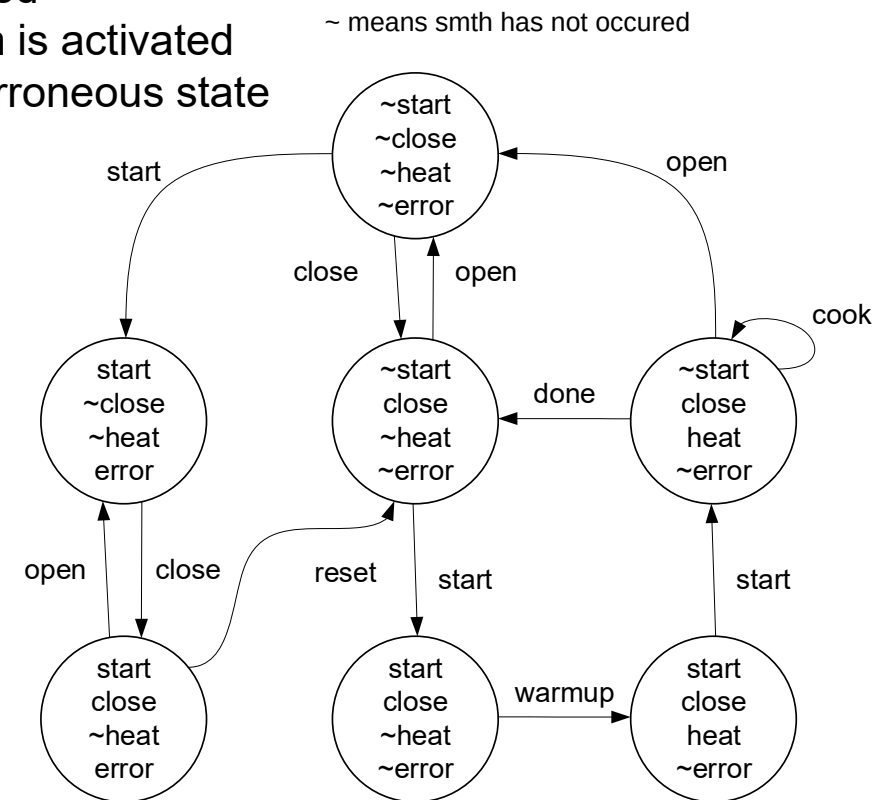
◆ System: Microwave Oven

First, the system is described using a state machine. Here, the state vector consists of three Boolean variables:

- `start` representing if the system is activated
- `close` representing if the lid is open or closed
- `heat` representing if the heating mechanism is activated
- `error` representing if the system is in an erroneous state

◆ Property: From every state, a start state can be reached using the start button.

- ◆ Is that a safe system?
- ◆ Is that formal verification?
- ◆ Do we have that model or do we want to create that model?





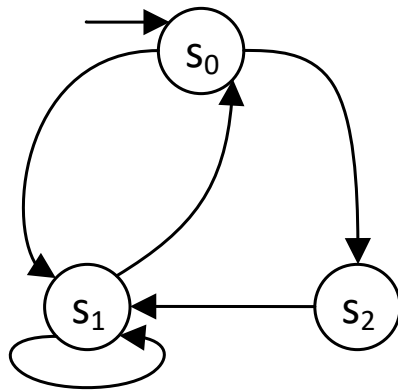
State space and variables

- ◆ Formal verification means **state space exploration**
- ◆ The state of a system, similar to the state of an object in object-oriented programming languages, consists of the current values of variables.
- ◆ There are dependent and independent variables.
 - **Independent variables** can change more or less freely between states. An example for such variables are inputs.
 - **Dependent variables** also can change between two states, but the changes depend on independent variables.
 - There can also be mixtures of both, variables that can change freely from state to state but sometimes also change as a reaction of inputs.



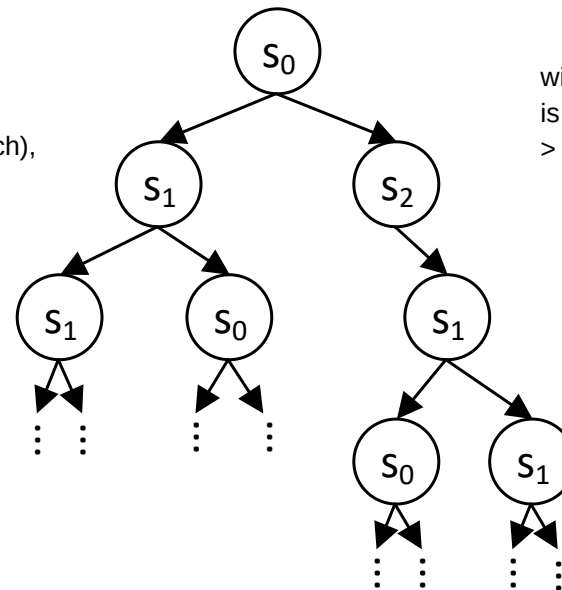
Computation Tree

- ◆ Let $T = \langle S, \rightarrow, s_0 \rangle$ be a transition system with
 - $S = \{s_0, \dots, s_n\}$ the states of the system,
 - $\rightarrow$ a set of transitions $\{\dots, (s_i, s_k), \dots\}$ and
 - $s_0 \in S$ an initial state.
- ◆ The computation tree is the acyclic unfolding of that transition system.
- ◆ Example:



if system is not final (endlich),
then checker will fail

A transition system



will end when no new information
is gained and loops are formed
> checker will know this and end
check

Computation tree



Propositional calculus

- ♦ The alphabet of the propositional calculus consists of a set of countable infinite variables V and operator symbols and structural signs O with
- ♦ $V = \{P_n \mid n \in N_0\}$ (N_0 : natural numbers incl. 0), e.g. $V = \{P_0, P_1, P_2, P_3, \dots\}$
- ♦ $O = \{\neg, \wedge, \vee, \rightarrow, \leftrightarrow, (,)\}$

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \rightarrow B$	$A \leftrightarrow B$
true	true	false	true	true	true	true
true	false	false	false	true	false	false
false	true	true	false	true	true	false
false	false	true	false	false	true	true

with propositional calculus, we cannot express specific properties in the computation tree. We can only express properties that can be checked for within every state (invariant).

NuSMV: `!, &, |, ->, <->, (, )`

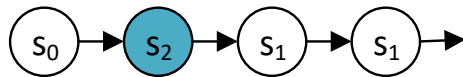
NuSMV: `INVARSPEC`



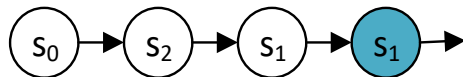
LTL – Linear(time) Temporal Logic

- ♦ LTL consist of formulas of the propositional calculus in addition with temporal operators. A LTL formula is true at a specific state i
- ♦ Overview of LTL connectives and examples for valid LTL formulas at state 0

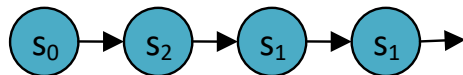
- $X \varphi$: φ is true in the next state



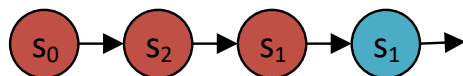
- $F \varphi$: φ is true at least once in the future



- $G \varphi$: φ is true for all states



- $\varphi_1 \cup \varphi_2$: φ_1 is true until φ_2 is true



LTL specifies what happens on all paths formulated for one specific path.

GF p means „For all paths, from every state, a state can be reached where p is true“.

So, p happens infinitely often.

NuSMV: LTLSPEC



CTL – Computation-Tree Logic

- ♦ CTL consist of formulas of the propositional calculus in addition with temporal operators with quantification over all paths.
- ♦ Overview
 - $AX \varphi$: On all paths, φ is true in the next state
 - **$EX \varphi$** : There is at least one path, where φ is true in the next state
 - $AF \varphi$: On all paths, φ is true at least once in the future
 - $EF \varphi$: There is at least one path, where φ is true at least once in the future
 - $AG \varphi$: On all paths, φ is true for all states
 - **$EG \varphi$** : There is at least one path, where φ is true for all states
 - $A \varphi_1 U \varphi_2$: On all paths, φ_1 is true until φ_2 is true
 - **$E \varphi_1 U \varphi_2$** : There is at least one path, where φ_1 is true until φ_2 is true

CTL specifies what happens on all paths from the initial state

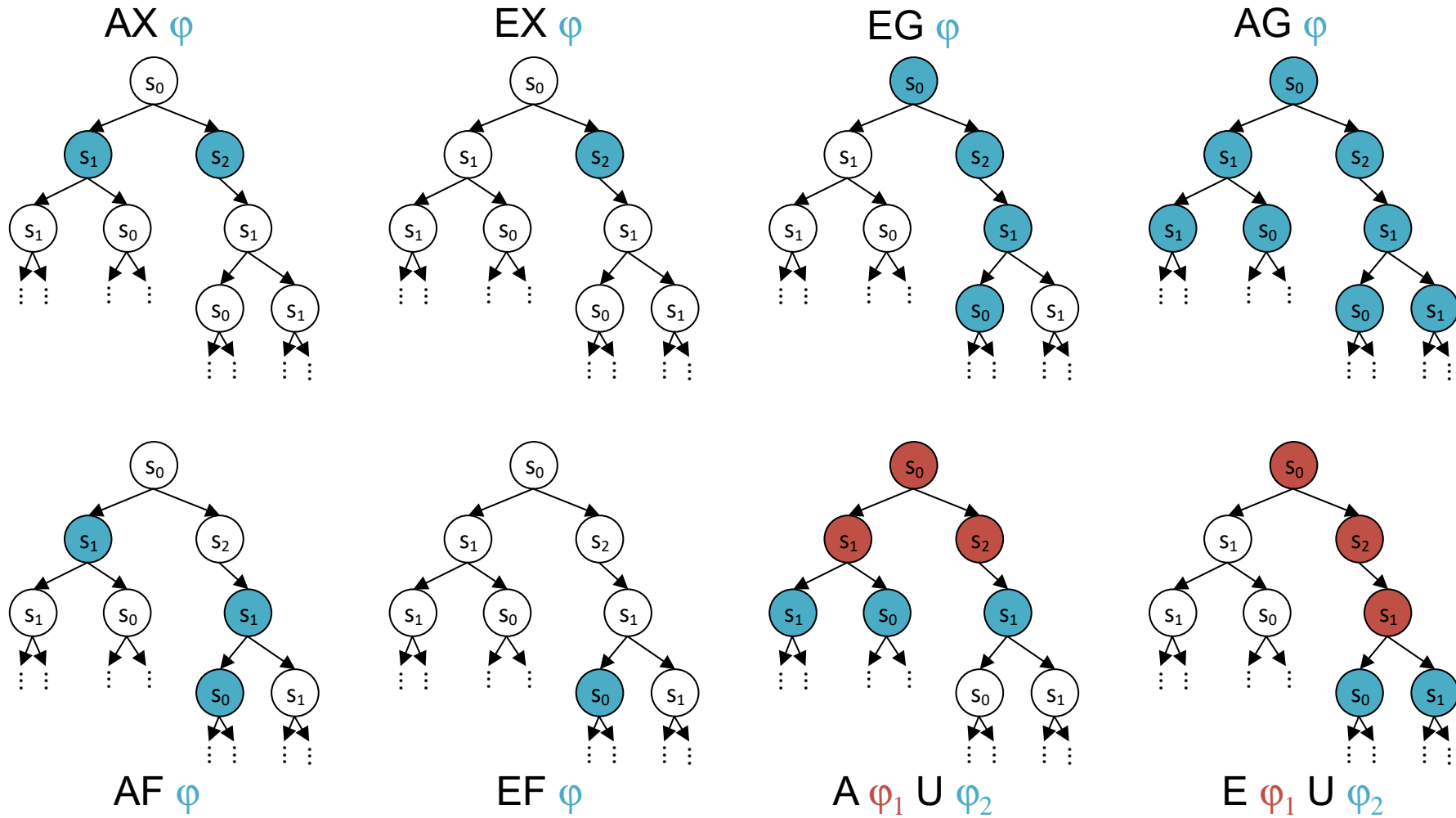
$AF p$ means on all paths from the initial state, p happens once!

Minimal set marked in bold

NuSMV : SPEC



CTL Examples





LTL and CTL are incomparable

- ♦ **CTL** can express something that cannot be expressed in LTL
“From every state, there exists a path that eventually reaches p.”

AG EF p

LTL can only express what happens on all paths. CTL is good at *branching-time inevitability/existence* (“no matter how choices branch, something eventually happens”).

- ♦ **LTL** can express something that cannot be expressed in CTL
“Along every execution path, p happens infinitely often.”

GF p

CTL can only express that on every path p is eventually true. LTL is good at *path-patterns over time* (“this keeps happening forever”, “infinitely often”, “eventually always”, etc.).



Typical properties to be checked

- ◆ **Liveness property**

From every state, i can reach a state where the system is operational

- ◆ **Reset property**

From every state, i can reach the initial state

- ◆ **Safety property**

Usually a condition that holds on all paths for all states, specifying things that never occur at the same time.

- ◆ **Security property**

On all paths, i never reach a state that requires authorization without prior authorization.



Model Checking Phases

- ◆ Model Checking is typically performed in three phases

- 1. Modelling phase**

The system to be analyzed is modeled using the formal model language of the model checking software. This is typically a state-based model. Additionally, the properties to be checked are also formulated in this language.

- 2. Model Checking phase**

The model checking software executes the model and checks the correctness of the model according to the properties.

- 3. Analysis phase**

If the system fulfills the properties, it is correct. If the system deviates from the properties, the model checker will find a *counterexample* that shows which steps have to be executed to transform the system from an initial state to an erroneous state.

Counterexamples can happen when

- a) Your model is wrong
- b) Your CTL / LTL are wrong
- c) Your system does not hold the condition
- d) Any of the above in combination

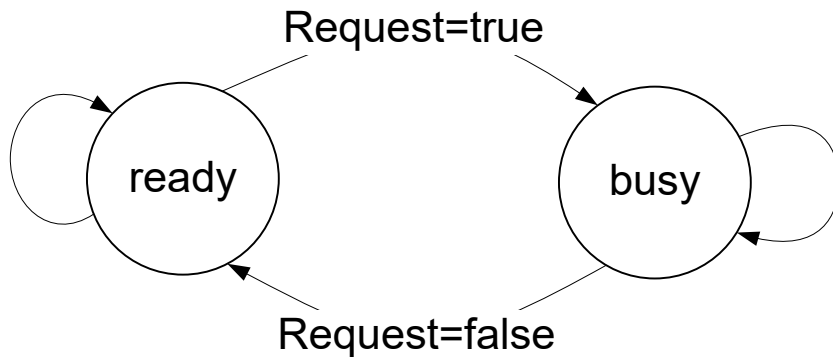




Finite-state Machines in NuSMV

- ◆ A finite state machine is a transition system that is in exactly one of a given number of finite states at any point in time with
 - a finite number of states and
 - a transition function

- ◆ Example



```
MODULE readybusy
VAR
  request : boolean;
  state : {ready, busy};
ASSIGN
  init(state) := ready;
  next(state) :=
    case
      state = ready & request : busy;
      state = busy & !request : ready;
      TRUE : state;
    esac;
SPEC AG(request -> AF state = busy)
```

```
bin\NuSMV.exe readybusy.smv
-- specification AG (request -> AF state = busy) is true
```



Traces in NuSMV (1)

- ♦ To check whether the automaton is specified correctly, NuSMV can create traces
- ♦ Example for readybusy.smv

```
> bin\NuSMV.exe -int readybusy.smv
NuSMV > go
NuSMV > pick_state -r
NuSMV > print_current_state -v
Current state is 1.1
request = FALSE
state = ready
NuSMV >
```

Interactive mode

Start

Select randomly a start state

Show active state

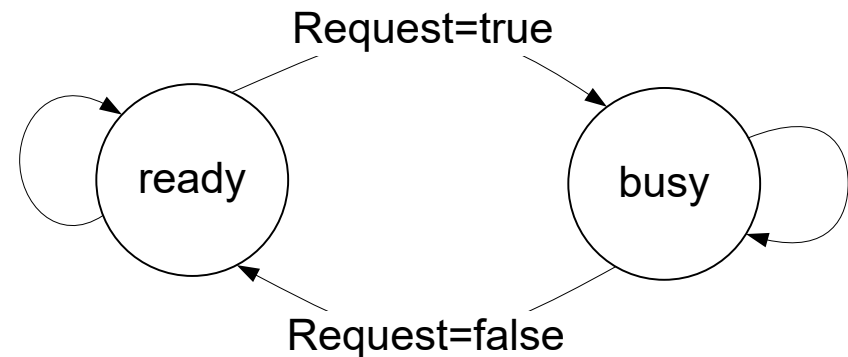


Traces in NuSMV (2)

```
NuSMV > simulate -r -k 3
** Simulation Starting From State 1.1 **
NuSMV > show_traces -v
  <!-- ### Trace number: 1 ### -->
Trace Description: Simulation Trace
Trace Type: Simulation
-> State: 1.1 <-
  request = FALSE
  state = ready
-> State: 1.2 <-
  request = TRUE
  state = ready
-> State: 1.3 <-
  request = FALSE
  state = busy
-> State: 1.4 <-
  request = TRUE
  state = ready
NuSMV >
```

Simulate randomly a trace with three steps

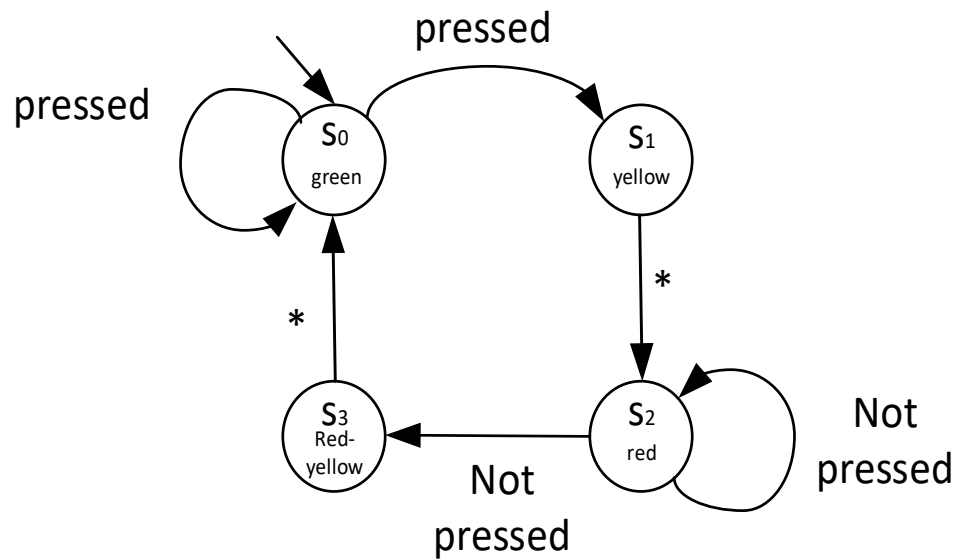
Print the trace



NuSMV has different interaction possibilities. More information can be found in <http://nusmv.fbk.eu/> NuSMV Tutorial & Documentation



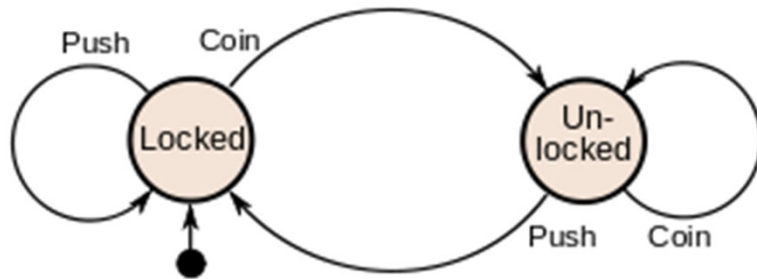
Exercise 1: Traffic Light



- ◆ Create a NuSMV Model and find some meaningful specifications to be checked



Exercise: A Turnstile



- 1) What are the variables of the state vector?
- 2) Which states can they take?
- 3) Is the automaton fully specified?
- 4) How does the system look like in NuSMV?
- 5) What properties are important?

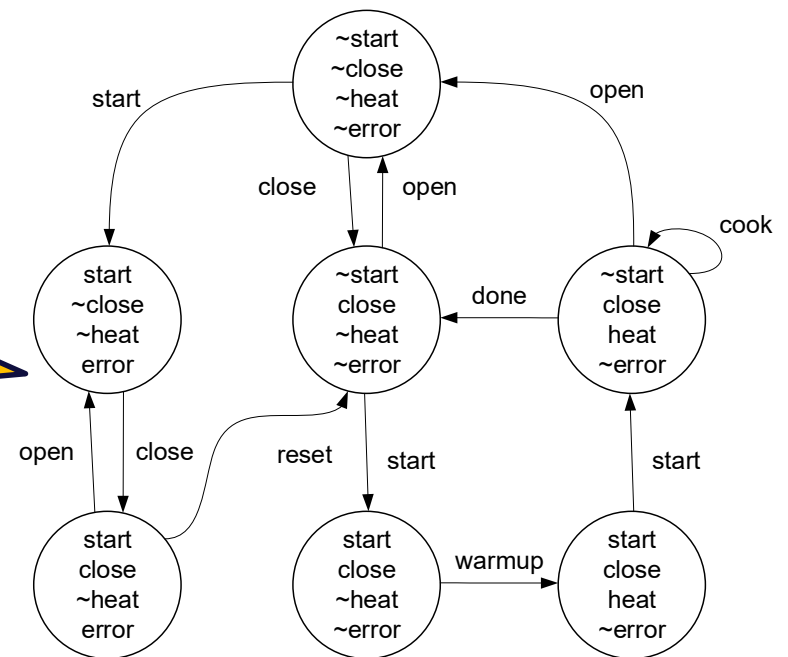


Task 2: Back to the Microwave Oven

1. Create a NuSMV model for the microwave oven.
2. Specify the variables as discussed.
3. Take design assumptions you know from a microwave oven and specify the behaviour of the variables
4. Create CTL properties to be checked



Do not use this automaton. Ist specified incompletely!



**Christel Baier and Joost-Pieter Katoen: Principles of model checking.*



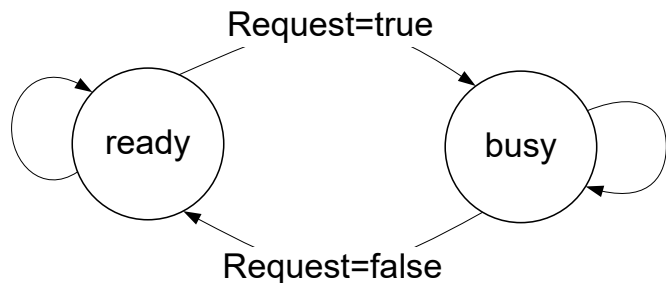
Modules: Example of a binary counter

```
MODULE counter_cell(carry_in)
  VAR
    value : boolean;
  ASSIGN
    init(value) := FALSE;
    next(value) := value xor carry_in;
  DEFINE
    carry_out := value & carry_in;
MODULE main
  VAR
    bit0 : counter_cell(TRUE);
    bit1 : counter_cell(bit0.carry_out);
    bit2 : counter_cell(bit1.carry_out);
```



Asynchronous Systems

- ◆ NuSMV is per se synchronous and time discrete. So, asynchronous systems need to be simulated using a scheduler variable.



```
MODULE readybusy(request)
VAR
  state : {ready, busy};
ASSIGN
  init(state) := ready;
  next(state) := case
    state = ready & request : busy;
    state = busy           : ready;
    TRUE                   : state;
  esac;

MODULE main
VAR
  sched : {processing, take_request};
  request : boolean;
  dispatch: {d1,d2};
  p1 : readybusy((dispatch=d1) & request);
  p2 : readybusy((dispatch=d2) & request);
ASSIGN
  init(sched) := take_request;
  next(sched) := {processing, take_request};

  init(request) := FALSE;
  next(request) := {TRUE, FALSE};

  init(dispatch) := d1;
  next(dispatch) := case
    p1.state=ready : d1;
    p2.state=ready : d2;
    TRUE : dispatch;
  esac;

FAIRNESS sched = processing
FAIRNESS sched = take_request
SPEC AG( request -> (p1.state = ready | p2.state = ready) )
```



Formal verification of AI-based Algorithms

- ♦ For a formal verification, you need to have a formal model. So, if the task is to mathematically prove that a neural network never fails an assumption under a certain condition, this condition needs to be formalized.

Formalization of input



easy

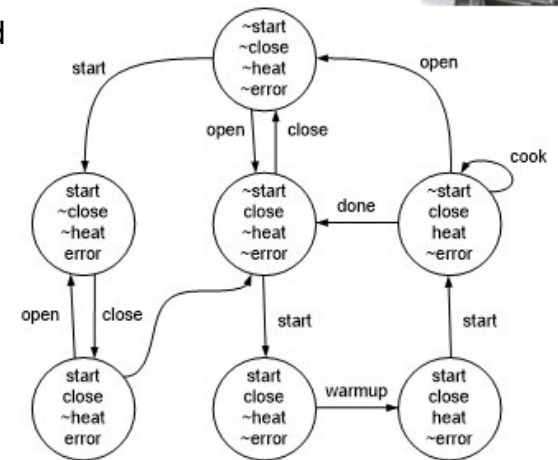
If you press start,

Formalization of output



hard

the microwave will turn on.

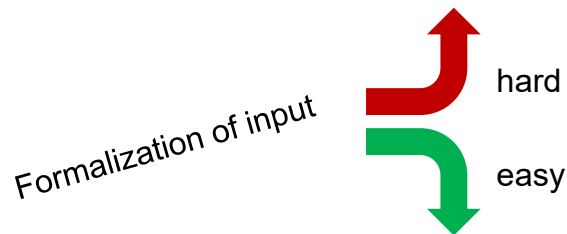




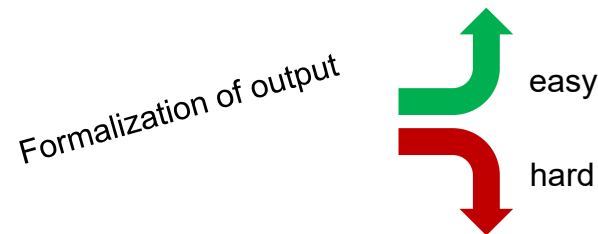
Formal verification of AI-based Algorithms

- ♦ For a formal verification, you need to have a formal model. So, if the task is to mathematically prove that a neural network never fails an assumption under a certain condition, this condition needs to be formalized.

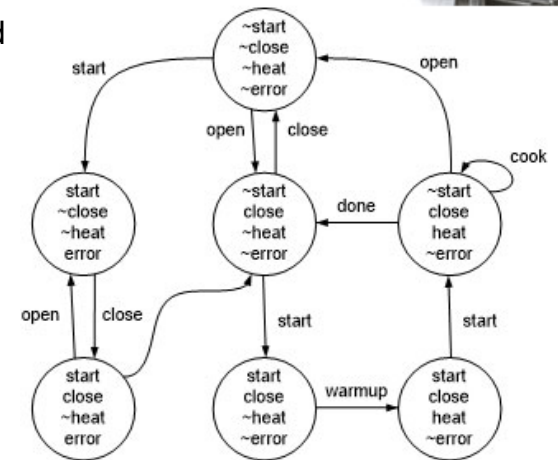
If the road bends to the left, the path planner never steers to the right.



If you press start,



the microwave will turn on.





Summary

- ◆ Formal verification requires a model, which can be an additional source for errors.
- ◆ Designing state machines for even simple systems can be complex tasks.
- ◆ Defining specifications to be checked is also an additional source for errors.
- ◆ The purpose of formal verification is not only the verification of properties, but also gaining a deeper understanding of the system itself.
- ◆ Formal verification enables us to proof properties, in contrast to testing, where we cannot proof certain properties.
- ◆ Here: only introduction, no parallel processing or time-aspects covered.